CSE563: Unit IV Study Notes

Graph Theory

School of Computer Science & Engineering

Session 2025–26

Contents

1 Unit IV: Graph Theory **2**
- 1.1 Graph Basics . . . 2
- 1.2 Graph Types . . . 3
- 1.3 Graph Traversing Algorithms: BFS and DFS . . . 4
- 1.4 Graph Searching Algorithms: Shortest Paths and Spanning Trees . . . 5
- 1.5 Graph Colouring . . . 7
- 1.6 Max-Flow / Min-Cut . . . 8
- 1.7 Unit IV Quick-Revision Sheet . . . 10

1 Unit IV: Graph Theory

Course Outcome CO4: analyse different graph theory concepts and use them in solving practical problems.

Road networks, social networks, the web, dependency graphs in a build system, state machines, neural-network computation graphs: whenever "things" are connected by "relationships", the mathematics is graph theory. This unit covers the vocabulary, the two fundamental traversals, shortest paths and spanning trees, colouring, and network flow.

1.1 Graph Basics

Definition 1.1. A **graph** $G = (V, E)$ consists of a finite set of **vertices** (nodes) V and a set of **edges** E, each edge joining two vertices. $|V| = n$ and $|E| = m$. In an **undirected** graph an edge is an unordered pair $\{u, v\}$; in a **directed** graph (digraph) it is an ordered pair (u, v), an *arc* from u to v.

Core Vocabulary

- **Adjacent** vertices share an edge; an edge is **incident** to its endpoints. A **loop** joins a vertex to itself; **parallel** (multiple) edges join the same pair.
- **Degree** $\deg(v)$ = number of edges incident to v (a loop counts twice). In a digraph: **in-degree** and **out-degree**. A vertex of degree 0 is *isolated*; degree 1 is a *pendant* (leaf).
- **Walk**: any sequence $v_0 e_1 v_1 \dots e_k v_k$ of consecutive incident edges. **Trail**: no repeated edge. **Path**: no repeated vertex. **Cycle**: closed path (returns to start, no other repeat). **Length** = number of edges.
- **Connected** (undirected): a path exists between every pair. The maximal connected pieces are **connected components**. A digraph is **strongly connected** if every vertex reaches every other by directed paths.
- **Subgraph**: $V' \subseteq V$, $E' \subseteq E$. **Spanning** subgraph: uses all of V.
- **Cut vertex** (articulation point): removal disconnects the graph. **Bridge**: an edge whose removal disconnects.

Formula & Mathematical Relation: Handshaking Lemma

$$\sum_{v \in V} \deg(v) = 2|E|. \tag{1}$$

Every edge contributes two to the degree sum. Consequence: the number of odd-degree vertices is **even**. In a digraph, $\sum \deg^-(v) = \sum \deg^+(v) = |E|$.

Worked Example: Handshaking

Can a graph have 7 vertices each of degree 3? Degree sum $= 21$, odd—impossible. A graph with degrees $3, 3, 2, 2, 2$ has $|E| = 12/2 = 6$ edges. In a network of 20 servers where every server links to exactly 4 others, there are $20 \times 4/2 = 40$ links.

Representations

- **Adjacency matrix** A $(n \times n)$: $a_{ij} = 1$ (or the weight) if edge $i \to j$, else 0. Symmetric for undirected graphs. $O(n^2)$ space; edge lookup $O(1)$; $(A^k)_{ij}$ counts walks of length k from i to j. Best for dense graphs.
- **Adjacency list**: for each vertex, a list of neighbours. $O(n + m)$ space; iterating neighbours

is proportional to degree. Best for sparse graphs (most real ones).

- **Incidence matrix** ($n \times m$): rows = vertices, columns = edges.

Example: $V = \{1,2,3\}$, $E = \{12, 23\}$: $A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$; lists: 1:[2], 2:[1, 3], 3:[2]. $A^2 = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \end{pmatrix}$: one walk of length 2 from 1 to 3 (via 2), and two closed walks of length 2 at vertex 2.

1.2 Graph Types

Type	Definition / key facts	CS example
Simple graph	No loops, no parallel edges. Max edges $\binom{n}{2}$.	Friendship network
Multigraph	Parallel edges allowed.	Multiple flights between cities
Directed (digraph)	Ordered edges.	Web links, Twitter follows, function calls
Weighted	Each edge has a number (cost, distance, capacity).	Road map, network latency
Complete K_n	Every pair adjacent; $\binom{n}{2}$ edges; each degree $n-1$.	Fully-meshed network
Regular (k-regular)	Every vertex has degree k. K_n is $(n-1)$-regular.	Peer-to-peer overlays
Bipartite	$V = X \cup Y$, every edge joins X to Y. $\iff$ no odd cycle $\iff$ 2-colourable. Complete bipartite $K_{m,n}$ has mn edges.	Users–items (recommendation), jobs–machines
Tree	Connected and acyclic. n vertices $\Rightarrow$ exactly $n-1$ edges; unique path between any two vertices; adding any edge creates exactly one cycle. A **forest** is a disjoint union of trees.	File systems, parse trees, decision trees
DAG	Directed acyclic graph. Always has a *topological order*.	Build dependencies, course prerequisites, task scheduling
Planar	Can be drawn with no crossing edges. Euler: $n - m + f = 2$ (connected). $m \leq 3n - 6$ for $n \geq 3$. K_5 and $K_{3,3}$ are **not** planar.	PCB / VLSI layout, map drawing
Eulerian	Has a closed trail using every edge once $\iff$ connected and all degrees even. *Euler path* (open) $\iff$ exactly 2 odd vertices.	Garbage-truck / snow-plough routing, DNA assembly
Hamiltonian	Has a cycle visiting every vertex exactly once. No simple test—NP-complete (Unit V). Dirac: $\deg(v) \geq n/2$ for all v suffices.	Travelling salesman

Table 1: Graph types. Note the asymmetry: Eulerian is easy to check (degrees), Hamiltonian is hard.

Worked Example: Recognising Types

- K_4: 6 edges, 3-regular, planar (draw it as a triangle with a centre vertex), Hamiltonian, *not* Eulerian (all degrees odd), not bipartite (has triangles).
- The cube graph Q_3: 8 vertices, 12 edges, 3-regular, bipartite (colour by parity of the number of 1-bits), planar, Hamiltonian.
- A path P_5 on 5 vertices: tree, bipartite, has an Euler *path* (two odd vertices), not Hamiltonian

cycle.

- Euler check on K_5: $n = 5$, $m = 10$; if planar, $m \leq 3(5) - 6 = 9$; but $10 > 9$, so K_5 is not planar.

Formula & Mathematical Relation: Trees: Equivalent Definitions

For a graph T with n vertices, the following are equivalent: (i) T is a tree (connected, acyclic); (ii) T is connected with $n-1$ edges; (iii) T is acyclic with $n-1$ edges; (iv) any two vertices are joined by exactly one path; (v) T is minimally connected (every edge is a bridge). A **spanning tree** of a connected G is a spanning subgraph that is a tree; G has one iff it is connected. A **rooted** tree has a distinguished root, giving parent/child/depth/height; a binary tree of height h has at most $2^{h+1} - 1$ nodes.

1.3 Graph Traversing Algorithms: BFS and DFS

Traversal = systematically visit every vertex reachable from a start vertex, each exactly once. Two strategies differ only in the data structure that holds the frontier.

Breadth-First Search (BFS)

Uses a **queue** (FIFO). Explores in "layers": all vertices at distance 1, then 2, ...

```
def BFS(G, s):
    visited = {s}; queue = [s]; dist = {s: 0}
    while queue:
        u = queue.pop(0)
        for v in G.adj[u]:              # in a fixed order
            if v not in visited:
                visited.add(v); dist[v] = dist[u] + 1
                queue.append(v)
```

Time $O(n+m)$ with adjacency lists. **Property:** in an unweighted graph BFS finds the *shortest path* (fewest edges) from s to every vertex; the tree of first-discovery edges is the BFS tree.

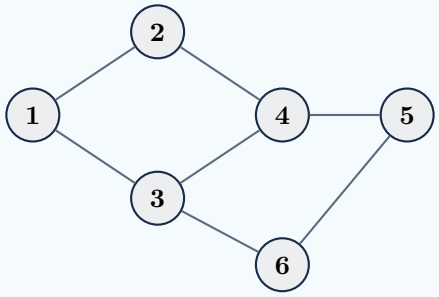
Depth-First Search (DFS)

Uses a **stack** (or recursion). Goes as deep as possible before backtracking.

```
def DFS(G, u):
    visited.add(u); pre[u] = time; time += 1
    for v in G.adj[u]:
        if v not in visited: DFS(G, v)
    post[u] = time; time += 1
```

Time $O(n+m)$. Edges are classified as *tree*, *back* (to an ancestor—signals a **cycle**), *forward* and *cross*. Post-order (finish) times give **topological sort** on a DAG: sort vertices by decreasing finish time.

Worked Example: BFS and DFS on the Same Graph



Adjacency lists (neighbours in increasing order): 1:[2, 3], 2:[1, 4], 3:[1, 4, 6], 4:[2, 3, 5], 5:[4, 6], 6:[3, 5].

BFS from 1. Queue [1]. Visit 1, enqueue 2, 3. Visit 2, enqueue 4. Visit 3: 4 already seen, enqueue 6. Visit 4, enqueue 5. Visit 6 (5 already queued). Visit 5.

Order: 1, 2, 3, 4, 6, 5. Distances: $d(2) = d(3) = 1$, $d(4) = d(6) = 2$, $d(5) = 3$. BFS tree edges: 1-2, 1-3, 2-4, 3-6, 4-5.

DFS from 1 (recursive). $1 \to 2 \to 4 \to 3$ (4's first unvisited neighbour) $\to 6 \to 5$; then 5's neighbours are done, backtrack all the way.

Order: 1, 2, 4, 3, 6, 5. Tree edges: 1-2, 2-4, 4-3, 3-6, 6-5. Back edges: 3-1, 5-4 (each proves a cycle exists).

Notice BFS says 5 is 3 hops from 1; DFS reached 5 by a path of length 5. BFS finds shortest paths, DFS does not.

Aspect	BFS	DFS
Data structure	Queue	Stack / recursion
Order	Level by level	Deep first, then backtrack
Memory	O(width): can be large	O(depth)
Finds	Shortest paths (unweighted), connected components, bipartiteness (2-colour by level parity)	Cycles, topological order, strongly connected components, articulation points, maze solving
Complete?	Yes (finite graphs)	Yes for finite graphs; may not terminate on infinite ones

Table 2: BFS vs DFS. Both are $O(n + m)$.

Worked Example: Topological Sort by DFS

Course prerequisites: $A \to C$, $B \to C$, $B \to D$, $C \to E$, $D \to E$. DFS in alphabetical order: start A: visit $A \to C \to E$; E finishes (1), C finishes (2), A finishes (3). Start B: $B \to$ (C seen) $\to D \to$ (E seen); D finishes (4), B finishes (5). Decreasing finish time: **B, D, A, C, E**. Check every edge goes left to right. ✓ (Kahn's algorithm—repeatedly remove a vertex of in-degree 0—gives an equally valid order such as A, B, C, D, E.)

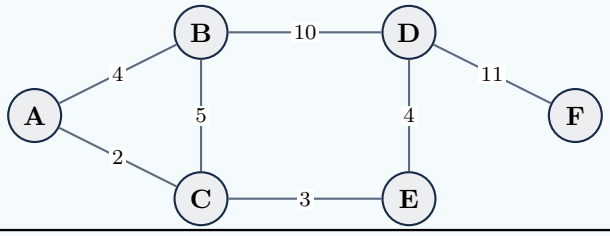
1.4 Graph Searching Algorithms: Shortest Paths and Spanning Trees

Traversal visits everything; *searching* finds something specific—a shortest route, a cheapest connecting network, a target state.

Dijkstra's Algorithm (single-source shortest paths, non-negative weights)

Maintain tentative distance $d[v]$ (initially ∞, $d[s] = 0$) and a min-priority queue. Repeatedly extract the unvisited vertex with the smallest d, mark it final, and **relax** each outgoing edge: if $d[u] + w(u, v) < d[v]$ then update $d[v]$ and set parent$[v] = u$. With a binary heap the time is $O((n + m) \log n)$. Greedy: once a vertex is extracted its distance is final—this is only valid because weights are non-negative.

Worked Example: Dijkstra from A



Step (extract)	A	B	C	D	E	F	Relaxations
init	**0**	∞	∞	∞	∞	∞	
A (0)	0	4	2	∞	∞	∞	B: 0+4, C: 0+2
C (2)	0	4	**2**	∞	5	∞	B: 2+5 = 7 $\not<$ 4; E: 2+3
B (4)	0	**4**	2	14	5	∞	D: 4+10
E (5)	0	4	2	9	**5**	∞	D: 5+4 = 9 < 14
D (9)	0	4	2	**9**	5	20	F: 9+11
F (20)	0	4	2	9	5	**20**	

Shortest distances: A=0, C=2, B=4, E=5, D=9, F=20. Path to F: follow parents $F \leftarrow D \leftarrow E \leftarrow C \leftarrow A$, i.e. A-C-E-D-F with cost $2 + 3 + 4 + 11 = 20$.

Exam Tip & Key takeaway: Which Shortest-Path Algorithm?

- Unweighted $\rightarrow$ **BFS**, $O(n + m)$.
- Non-negative weights $\rightarrow$ **Dijkstra**.
- Negative weights (no negative cycles) $\rightarrow$ **Bellman–Ford**, $O(nm)$; also detects negative cycles.
- All pairs $\rightarrow$ **Floyd–Warshall**, $O(n^3)$ dynamic programming (Unit VI).
- With a heuristic (games, maps) $\rightarrow$ **A***: Dijkstra ordered by $d[v] + h(v)$.

Dijkstra fails with negative edges because it finalises a vertex too early. Example: $s \rightarrow a$ (1), $s \rightarrow b$ (5), $b \rightarrow a$ (-10): Dijkstra finalises a at 1 but the true distance is -5.

Minimum Spanning Tree (MST)

Given a connected weighted graph, find a spanning tree of minimum total weight. Two greedy algorithms, both correct by the *cut property* (the lightest edge crossing any cut belongs to some MST):

- **Kruskal:** sort edges by weight; add each edge unless it creates a cycle (union–find). $O(m \log m)$.
- **Prim:** grow a tree from any vertex; always add the cheapest edge leaving the tree. $O(m \log n)$ with a heap. Like Dijkstra but keyed on edge weight, not path length.

Worked Example: Kruskal on the Dijkstra Graph

Sorted edges: AC 2, CE 3, AB 4, ED 4, BC 5, BD 10, DF 11. Take AC ✓, CE ✓, AB ✓, ED ✓, BC × (B and C already connected), BD × (cycle), DF ✓. Five edges for six vertices. MST weight $= 2 + 3 + 4 + 4 + 11 = 24$.
Prim from A picks $AC(2)$, $CE(3)$, $AB(4)$, $ED(4)$, $DF(11)$—the same tree. (The MST is unique when all weights are distinct; here the two 4s do not create a tie that matters.)

Common Pitfall & Misconception: Shortest-Path Tree ≠ MST

The shortest-path tree minimises *each path from the root*; the MST minimises *total edge weight*. Triangle with $AB = 2$, $BC = 2$, $AC = 3$: the MST is $\{AB, BC\}$ (weight 4). Dijkstra from A picks AB (2) and AC (3), because the path A-B-C costs $4 > 3$. Different trees. Never assume one gives the other. (On the worked graph above they happen to coincide except that the shortest-path tree is rooted.)

1.5 Graph Colouring

Definition 1.2. A **proper vertex colouring** assigns each vertex a colour such that adjacent vertices get different colours. The **chromatic number** $\chi(G)$ is the minimum number of colours needed. A graph with $\chi(G) \leq k$ is *k-colourable*.

Facts and Bounds

- $\chi(K_n) = n$. $\chi(\text{tree}) = 2$ (for $n \geq 2$). $\chi(C_n) = 2$ if n even, 3 if odd. $\chi(G) = 1$ iff G has no edges.
- $\chi(G) = 2 \iff G$ is bipartite (and has an edge) $\iff$ no odd cycles. Check with BFS: colour by level parity, fail if an edge joins same-parity vertices.
- Lower bound: $\chi(G) \geq \omega(G)$, the size of the largest clique (complete subgraph).
- Upper bound: $\chi(G) \leq \Delta(G) + 1$ where Δ is the maximum degree (greedy achieves this). *Brooks:* $\chi \leq \Delta$ unless G is complete or an odd cycle.
- **Four Colour Theorem:** every planar graph is 4-colourable (so every map can be coloured with 4 colours).
- Deciding whether $\chi(G) \leq 3$ is **NP-complete** (Unit V); in practice we use greedy heuristics.

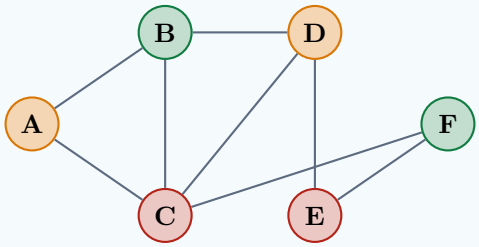
Formula & Mathematical Relation: Greedy Colouring (Welsh–Powell)

1. Order vertices by decreasing degree.
2. Take the first uncoloured vertex, give it a new colour, and give the same colour to every later uncoloured vertex not adjacent to any vertex already holding that colour.
3. Repeat with the next colour until all vertices are coloured.

Uses at most $\Delta + 1$ colours; not always optimal, but fast ($O(n^2)$ or $O(n + m)$ with care).

Worked Example: Exam Timetabling by Colouring

Six courses; an edge means some student takes both (so they cannot be examined in the same slot).



Degrees: A=2, B=3, C=4, D=3, E=2, F=2. Triangle $A, B, C \Rightarrow \chi \geq 3$.
Welsh–Powell order: C, B, D, A, E, F.

- **Colour 1 (red):** C. Then B? adjacent to C, skip. D? adjacent, skip. A? adjacent, skip. E: not adjacent to $C \to$ red. F: adjacent to C, skip. Red $= \{C, E\}$.
- **Colour 2 (green):** B. D adjacent to B, skip. A adjacent, skip. F: not adjacent to $B \to$ green. Green $= \{B, F\}$.
- **Colour 3 (gold):** D, then A (not adjacent to D). Gold $= \{D, A\}$.

Three time slots suffice and three are necessary: $\chi = 3$. Slot 1: $\{C, E\}$; Slot 2: $\{B, F\}$; Slot 3: $\{A, D\}$.

Where Computer Science Uses This: Colouring in Systems

Register allocation: variables are vertices; an edge joins two that are live at the same time; colours are CPU registers. **Frequency assignment:** transmitters that interfere must get different channels. **Sudoku** is 9-colouring of an 81-vertex graph. **Scheduling** with conflicts (exams, sports fixtures). **Edge colouring** (adjacent edges differ) models round-robin tournaments; Vizing: $\Delta \leq \chi'(G) \leq \Delta + 1$.

1.6 Max-Flow / Min-Cut

Definition 1.3. A **flow network** is a digraph with a **source** s, a **sink** t, and a non-negative **capacity** $c(u, v)$ on each arc. A **flow** f assigns $f(u, v)$ to each arc such that

- **capacity constraint:** $0 \leq f(u, v) \leq c(u, v)$;
- **conservation:** for every $v \notin \{s, t\}$, flow in = flow out.

The **value** $|f|$ is the net flow out of s. An s–t **cut** (S, T) partitions V with $s \in S$, $t \in T$; its **capacity** is $\sum_{u \in S, v \in T} c(u, v)$ (only arcs going from S to T count).

Formula & Mathematical Relation: Max-Flow Min-Cut Theorem

$$\max_{f} |f| \;=\; \min_{(S,T)} c(S, T). \tag{2}$$

Any flow is at most any cut (weak duality); Ford–Fulkerson shows equality is achieved. The three statements are equivalent: (i) f is a maximum flow; (ii) the residual network has no augmenting path; (iii) $|f| = c(S, T)$ for some cut.

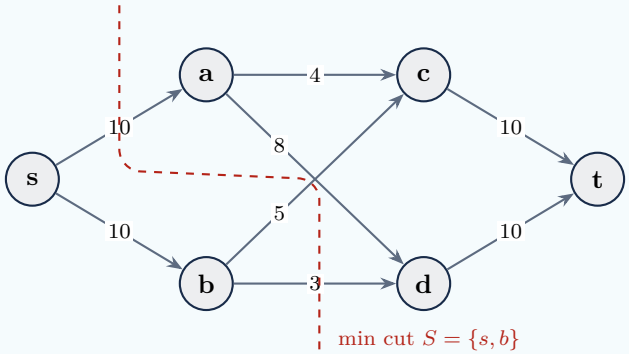
Ford–Fulkerson Method

1. Start with $f = 0$.
2. Build the **residual network**: arc $u \to v$ with residual capacity $c(u, v) - f(u, v)$ if positive, and a **reverse arc** $v \to u$ with capacity $f(u, v)$ if positive (this lets a later path *undo* earlier flow).

3. Find any $s \to t$ path in the residual network (an **augmenting path**); push its bottleneck (minimum residual capacity) along it. Repeat until no path exists.

With BFS choosing the shortest augmenting path (**Edmonds–Karp**) the running time is $O(nm^2)$; with arbitrary paths and irrational capacities it may not terminate. The min cut is read off at the end: S = vertices reachable from s in the final residual network.

Worked Example: Maximum Flow and Minimum Cut



Augmenting paths (bottleneck in brackets):

1. $s \to a \to c \to t$ [4]: residuals sa:6, ac:0, ct:6. Flow 4.
2. $s \to a \to d \to t$ [6] (sa has only 6 left): sa:0, ad:2, dt:4. Flow 10.
3. $s \to b \to c \to t$ [5]: sb:5, bc:0, ct:1. Flow 15.
4. $s \to b \to d \to t$ [3]: sb:2, bd:0, dt:1. Flow 18.

Residual search: from s the only forward arc with capacity left is $s \to b$ (2). From b: $bc = 0$, $bd = 0$, and the only reverse arc is $b \to s$. Nothing reaches t: no augmenting path. **Max flow = 18.**

Min cut: $S = \{s, b\}$ (reachable in the residual network), $T = \{a, c, d, t\}$. Arcs from S to T: $s \to a$ (10), $b \to c$ (5), $b \to d$ (3). Capacity = 18 = max flow. ✓ Note that arc $a \to$ anything is inside T and does not count; nor would any arc from T back into S.

Final flows: sa=10, sb=8, ac=4, ad=6, bc=5, bd=3, ct=9, dt=9. Conservation at a: in 10, out $4 + 6$. ✓

Exam Tip & Key takeaway: Cut Capacity Counts Only $S \to T$ Arcs

Students lose marks by adding arcs that go from T back to S, or arcs internal to S. Also: there may be several minimum cuts; any one of capacity equal to the max flow is correct. A quick way to *verify* a claimed max flow is to exhibit a cut of the same capacity.

Where Computer Science Uses This: Applications of Flow

- **Maximum bipartite matching** (jobs to workers, students to projects): add $s \to$ every left vertex and every right vertex $\to t$, all capacities 1; max flow = size of maximum matching.
- **Network reliability / bottleneck detection:** the min cut is the set of links whose failure disconnects the most throughput.
- **Image segmentation** (graph cuts): pixels are vertices, s = foreground, t = background; the min cut is the object boundary.
- **Project selection, baseball elimination, airline scheduling**—all reduce to max flow.

1.7 Unit IV Quick-Revision Sheet

Exam Tip & Key takeaway: Ten Things to Carry Into the Exam

1. $\sum \deg(v) = 2|E|$; number of odd-degree vertices is even.
2. Tree: connected + acyclic $\iff$ connected with $n-1$ edges. Planar: $n - m + f = 2$, $m \leq 3n - 6$.
3. Eulerian circuit $\iff$ all degrees even (connected); Euler path $\iff$ exactly two odd vertices. Hamiltonian: no easy test.
4. Bipartite $\iff$ no odd cycle $\iff$ 2-colourable (BFS parity test).
5. BFS = queue = shortest unweighted paths; DFS = stack = cycles, topological sort. Both $O(n+m)$.
6. Dijkstra: extract-min, relax; non-negative weights only. Negative weights $\rightarrow$ Bellman–Ford.
7. MST: Kruskal (sort edges, union–find) or Prim (grow from a vertex). Cut property justifies both.
8. $\omega(G) \leq \chi(G) \leq \Delta(G) + 1$; planar $\Rightarrow \chi \leq 4$; Welsh–Powell = greedy by decreasing degree.
9. Max flow = min cut. Ford–Fulkerson: augment along residual paths (include reverse arcs!). Edmonds–Karp $O(nm^2)$.
10. Cut capacity counts only $S \rightarrow T$ arcs; min cut S = vertices reachable from s in the final residual network.